Pointwise Bidirectional RGB-D: Exact Implementation Specification

Codex context, production interfaces, CUDA modules, numerical tests and mandatory visual debugging

Implementation Contract

22 October 2026

Contents

0. READ THIS FIRST — FINAL ALGORITHM, NO REINTERPRETATION **2**

1. INSPECT REPOSITORY AND PROTECT OLD CODE **3**

2. HARD DATA CONTRACT **3**
- 2.1 Per-frame inputs . . . 3
- 2.2 Stable observation identities . . . 4
- 2.3 Point identity and storage . . . 4

3. FRAME-0 INITIALIZATION — ALL FOREGROUND POINTS SOLVED **4**

4. THE ONLY PER-POINT DYNAMIC STATE IS SIX-DIMENSIONAL **4**
- 4.1 Correct reverse-oriented process factors . . . 5

5. EXACT POINT-ONLY RGB-D OBSERVATION **5**

6. MATCHING AGAINST THE FROZEN TRANSPORTED GEOMETRY **5**

7. FIRST AND ONLY LARGE OPTIMIZATION (INITIAL POINTS) **6**

8. INDIVIDUAL UNMATCHED POINT -> BIDIRECTIONAL FULL-SEQUENCE OPTIMIZATION **6**
- 8.1 Seed and unique identity . . . 6
- 8.2 Backward branch, frame by frame . . . 7
- 8.3 Forward branch, frame by frame . . . 7
- 8.4 Commit new SOLVED point, then repeat . . . 7

9. EXACT HIGH-LEVEL CONTROLLER (PSEUDOCODE) **7**

10. REQUIRED NUMERICAL + VISUAL DEBUG (BLOCKING DELIVERABLE) **8**

11. TESTS THAT MUST PASS **9**

12. IMPLEMENTATION MODULE CONTRACT **10**

13. DELIVERY ORDER AND DEFINITION OF DONE **11**

14. PRIOR DOCUMENTS AND PRECEDENCE **11**

Date: 22 October 2026
Authority: Normative. Implement *all* operations, deliver the mandated debugging and tests, and report any data/observability limitations.
Primary source repository: https://github.com/SamueleMara/RigEuler3DGS — inspect current code and reuse mathematical or data-loading conventions, but implement the new **point-only** pipeline as an isolated module. Do not modify the proven static masked 3DGS training/rasterizer or make old stages regress.
Companion deliverables: `Pointwise_Bidirectional_RGBD_Theory_2026-10-22.pdf` and `Pointwise_Bidirect 10-22.pdf`.

0. READ THIS FIRST — FINAL ALGORITHM, NO REINTERPRETATION

The user requires **one large initial forward optimization** for all frame-0 foreground XYZ/RGB points; then **one unmatched RGB-D observation at a time** triggers a fresh point with its **own 6D position+velocity state**, initialized at the actual discovery frame. Optimize **this point alone** frame-by-frame **backward to frame 0**, and separately **forward to final frame T**, with the same IEKF (or separate UKF mode) observation and process model. **Previously optimized SOLVED trajectories are frozen, never re-optimized when a new point appears.** Insert the new point and its bidirectional trajectory into the SOLVED geometry, refresh matching, restart the backward unmatched search from T, repeat.

```
Initial masked RGB-D frame 0 -> ALL valid foreground XYZ/RGB observations SOLVED
            |
            v
ONE LARGE vectorized free-point IEKF forward t=0 ... T
            |
      Cache old tracks, covariances, observation ownership (FREEZE)
            |
            v
SCAN raw foreground RGB-D geometry t=T ... 0
  Compare to transported, already SOLVED positions using kNN + visibility
            |
    first genuinely unmatched observation (t_d,c,u) -> NEW POINT SEED
            |
         anchor x(t_d)=backprojected actual RGB-D observation
            |
  optimize ONLY the new point: t_d -> 0 and t_d -> T
            |
      store full state/covariance/real-vs-predicted observation history
            |
       incorporate new point into SOLVED; no separate validation phase
            |
  update correspondence/coverage, restart scan at T; repeat
```

Forbidden architectural substitutions:

- No per-point `SE(3)` 6-DoF pose, rotation, angular velocity, 12D state, Gaussian shape, opacity, alpha compositor, point surfel normal, or per-point rigid-part assignment. A point has position and linear velocity only. Its three-dimensional displacement can represent any trajectory, including curved tracks of a rotating object.
- No moving/static part clustering, `K` inference, universal static motion forcing, supernodes, rigid consensus, neighbor-motion weighted trajectories, or group transport in this implementation.
- No global re-optimization after adding points; **old states and covariances remain bitwise frozen** throughout incremental discovery. No separate candidate "confirmation" round, no withholding a new point until some arbitrary number of measurements exists.
- No global IEKS/RTS batch smoother disguised as frame-wise bidirectional IEKF; no double counting

observation information; no synthetic RGB/depth/background measurements outside the original foreground mask.
- No treating previously optimized trajectory estimates as fresh independent measurements. They are cached positions and matching context, not new evidence in the new point's Kalman gain.
- No sample-a-few-seeds approximation for the first implementation. **Each unmatched valid RGB-D observation is individually eligible as a seed**, processed deterministically one seed at a time. Efficient indexing is required; clustering/batching may be a future optimization, not a silent change of algorithm.
- No declaring success without numerical **and** visual debug artifacts from real data.

Meaning of SOLVED: a persistent point identity with a completed bidirectional *estimate* and a known real source observation. It does **not** assert that every frame is observed. Predicted-only temporal positions carry increasing uncertainty; they must not be shown as real measurements. A point encountered only once is still seeded and optimized over the whole temporal range; the lack of temporal evidence is clearly reported instead of inventing motion. On numerical failure, report a failed discovery attempt without pretending a trajectory was computed, and retry only when the data/model version changes.

1. INSPECT REPOSITORY AND PROTECT OLD CODE

1. At runtime locate the actual source tree; do not assume a hard-coded Ubuntu home path. Record git commit hash, compiler/CUDA/Torch versions and camera dataset paths.
2. Read `tools/run_iekf_scene.py` for frames/cameras/timestamps/ground-truth segregation; `estimator/state.py`, `process.py`, `covariance.py`, `sparse_iekf.py` for filter math; `estimator/masked_observation.py` for object-only depth; `docs/PIPELINE_CODE_MAP.md` for active stages; `debug/` for viewer patterns.
3. Existing `tools/kpart_pipeline.py::bootstrap_positions` resets `P` on each frame and forces `use_supernodes=True`. DO NOT copy either error. Its old 12D state is unsuitable for point-only tracking.
4. Create an isolated package, e.g. `pointwise_rgbd/`, new runner `tools/run_pointwise_rgbd_discovery.py`, separate unit/integration tests. Existing Gaussian training, existing output folders and CUDA rasterizers remain unchanged.
5. If a point-only project already exists in the checkout, inspect and adapt it instead of creating a duplicate framework; preserve the normative semantics below.

2. HARD DATA CONTRACT

2.1 Per-frame inputs

For chronological frame `t=0,...,T` and synchronized camera `c=0,...,C-1`:

- Raw native RGB `I[t,c]` in `[0,1]^3`; raw metric Z-depth `D[t,c]`; native whole-object binary mask `M[t,c]` and real return validity `Vraw[t,c]`.
- Intrinsics `(fx,fy,cx,cy)` with appropriate calibrated distortion treatment; immutable world-to-camera `(Rcw,tcw)`; monotonically increasing real timestamp `tau[t]` in seconds, or explicit declared physical frame rate if timestamps unavailable.
- A geometrically observed foreground pixel iff `M && Vraw && isfinite(D) && D>0`. Do not fill depth outside. Do not turn mask-zero RGB into a measurement. Do not generate artificial background color/depth.
- Ensure depth convention: camera optical-axis Z, not Euclidean range unless converted. Validate camera-pose direction, axis/handedness, calibration units, pixel-center convention, distortion handling and depth alignment with RGB/mask.
- Ignore dataset ground-truth moving-part ids, joints, displacements, object pose when estimating. GT may be loaded only in a separately identifiable **post-run** evaluation path.

2.2 Stable observation identities

One raw observation has immutable ID `(original_frame, camera_id, linear_pixel_id)` plus observed RGB and metric depth. Do not duplicate observations in different buffers and inadvertently count them twice. Maintain statuses `UNCLAIMED`, `OWNED(point_id)`, `EXAMINED_FOR_MODEL_VERSION`, and, only during a single candidate fit, temporary reservation. An observation can be owned by **at most one** point; releases/commits are atomic and audit logged. Multiple camera observations of a physical point have different observation IDs and may be jointly used once each.

2.3 Point identity and storage

`point_id` is stable and never reused. Store `anchor_observation_id`, `anchor_frame`, `anchor_xyz_world`, appearance RGB (for **matching only**), and temporal arrays `x[T+1,3]`, `v[T+1,3]`, `P[T+1,6,6]`, `observed[T+1]`, `measurement_refs[frame]`, `innovation_stats[frame]` and provenance. Old entries are immutable after their initial complete optimization. A 6D state has **no** orientation, Gaussian intrinsic shape or Lie-group reset.

3. FRAME-0 INITIALIZATION — ALL FOREGROUND POINTS SOLVED

1. Enumerate every valid frame-0 foreground observation of all cameras, backproject to camera XYZ and transform to world XYZ using inverse extrinsics.
2. Treat **all** such geometry as initially SOLVED. You may deterministically deduplicate truly duplicate cross-camera samples using measurement covariance and provenance, but do not discard foreground observations through downsampling, inferred motion classes or arbitrary sparse seed quotas. Keep every original observation ID mapped to its representative point.
3. For point `i`: nominal `s_i,0=[p_i; 0]` (zero is a neutral *initial estimate*, not an assertion of static); initial position covariance from the sensor's backprojection uncertainty; initial velocity prior with declared physical variance. Do not set zero covariance in unobserved velocity dimensions.
4. Record precisely how each frame-0 observation was fused, and assert complete initial foreground coverage under the explicitly declared fusion semantics.

4. THE ONLY PER-POINT DYNAMIC STATE IS SIX-DIMENSIONAL

$$s_{i,t} = \begin{bmatrix} x_{i,t} \\ v_{i,t} \end{bmatrix} \in \mathbb{R}^6, \quad F(\Delta t) = \begin{bmatrix} I_3 & \Delta t I_3 \\ 0 & I_3 \end{bmatrix}.$$

For world position/velocity and white-acceleration spectral density matrix `Qc` (units $m^{2/s}3$), and optional **separately declared** direct position random-walk spectral density `Qx`:

$$s_{t+1}^- = F(\Delta t) s_t^+, \quad P_{t+1}^- = F P_t^+ F^T + Q_f(\Delta t),$$

$$Q_f(\Delta t) = \begin{bmatrix} \frac{\Delta t^3}{3} Q_c + \Delta t Q_x & \frac{\Delta t^2}{2} Q_c \\ \frac{\Delta t^2}{2} Q_c & \Delta t Q_c \end{bmatrix}.$$

Use the same actual positive interframe duration throughout. Do **not** reset `P` per frame. Preserve all 6x6 cross-blocks. In prediction-only intervals, propagate full uncertainty. Use float64 covariance solves or numerically proven stable factorization, symmetry/PSD checks, and diagnostics for every fallback/jitter. Default process-noise values must be explicitly declared, physically interpretable, and reported, not magic tunings hidden in CUDA.

4.1 Correct reverse-oriented process factors

For propagating from `t+1` to `t`, use `F_rev=F(dt)^-1=[[I,-dt I],[0,I]]`; **physical velocity is not negated**. Under the *same forward factor*, the algebraically equivalent reverse residual has covariance

$$Q_{rev} = F_{rev} Q_f F_{rev}^T = \begin{bmatrix} \frac{dt^3}{3} Q_c + dt Q_x & -\frac{dt^2}{2} Q_c \\ -\frac{dt^2}{2} Q_c & dt Q_c \end{bmatrix}$$

for the adopted model (the optional independent position random walk simply adds `dt Qx` to position-position). Use this reverse *process factor* as the model for the backward branch anchored at discovery time. Do not mislabel it as the exact reverse conditional of an arbitrary forward-filter posterior; that conditional needs a joint temporal prior. Verify the quadratic factor identity to numerical precision for arbitrary state pairs.

5. EXACT POINT-ONLY RGB-D OBSERVATION

One associated pixel supplies `z=[u,v,Z]`, once, and appearance `I` assists association but does not generate an extra state residual unless an explicit calibrated photometric model is added (not in v1).

World-to-camera `q=Rcw x+tcw=(X,Y,Z)`, ideal pinhole output

$$h_c(s) = \begin{bmatrix} f_x X/Z + c_x \\ f_y Y/Z + c_y \\ Z \end{bmatrix}, \quad H_c = \begin{bmatrix} f_x/Z & 0 & -f_x X/Z^2 \\ 0 & f_y/Z & -f_y Y/Z^2 \\ 0 & 0 & 1 \end{bmatrix} R_{cw} \begin{bmatrix} I_3 & 0_3 \end{bmatrix}.$$

Calibrated lens distortion modifies the projection and Jacobian consistently. `Rz` is the declared **joint measurement covariance** in pixel/pixel/metres coordinates; may depend on real depth, camera, mask quality. Color is matching evidence. Do **not** independently add a world XYZ residual from this same pixel after using `(u,v,Z)`; that double-counts depth. If one instead uses world XYZ, propagate its full covariance through backprojection and use it *instead of*, not in addition to, the source `(u,v,Z)` observation.

Association and ownership are conditional/discrete; the Gaussian IEKF is a **local conditional approximation**, not an exact globally marginalized correspondence posterior.

6. MATCHING AGAINST THE FROZEN TRANSPORTED GEOMETRY

For each frame `t`, build a CUDA-friendly spatial index of the positions `x_i,t` of all currently SOLVED points; equivalently project visible points per camera with measured-depth visibility checks. Match candidate valid foreground observations using spatial nearest-neighbor candidate search and **camera-ray compatibility**; 3D proximity without correct image-ray/occlusion compatibility is insufficient. Exclude invalid depth/outside masks. Enforce one-to-one ownership of *raw observation IDs*.

For candidate point i / observed backprojected world sample y, let `S_xy = P_xx,i,t + Sigma_y` (conditionally independent approximation, clearly labeled); Mahalanobis score `d2=(y-x)^T S_xy^-1 (y-x)`. For pixel-based matching, use `r=z-h(s)` and innovation `S=H P H^T+R`. Do not use both forms as two likelihood terms. Image RGB consistency may disambiguate appearance but never override an impossible geometric match. Derive nominal gate from declared noise/specified tail probability; measure and plot real matched/unmatched distributions **and their gap** on data. Never assert such a gap without measuring it. If ambiguous, log it and let it remain unmatched/discoverable under the declared association policy.

Important difference: a **sampled point cloud** is not a continuous rendered surface. A point can be spatially near another surface without corresponding to its measurement; conversely a sampled surface

contains finite spacing between points. Report sample-spacing distribution and account for it in matching, without inventing a Gaussian footprint or synthetic fill. Do not mark foreground pixels as explained by a projected high-uncertainty point without a compatible real correspondence. Prediction-only poses do not have direct observations.

Search order: scan `t=T,T-1,...,0`; within a frame, stable camera order and deterministic pixel order. The *first unmatched valid observation* creates the next individual seed; no unsolved patches or K-part clustering. All unmatched observations are eligible across repeated scans.

7. FIRST AND ONLY LARGE OPTIMIZATION (INITIAL POINTS)

At t=0, initialize the posterior from original real RGB-D measurements **once**. If initial point coordinates and position covariance were already obtained by backprojection/fusion, do **not** assimilate the same raw observations again as independent likelihood factors. For t=1,...,T:

1. For ALL initial points in vectorized GPU chunks, predict `s^-`, `P^-` from previous **posterior** (not reconstructed prior).
2. Project predictions into all calibrated cameras and find valid foreground association candidates against measured `(u,v,Z)`; preserve consistent raw-observation ownership. Associations remain fixed within a single IEKF inner solve; re-associate only between accepted outer linearizations if implemented, with audit trail.
3. For each point with a fixed association set `A_i,t`, perform Euclidean 6D **iterated** EKF: minimize

$$E_i(s) = \frac{1}{2}\|s - s^-\|^2_{(P^-)^{-1}} + \frac{1}{2}\sum_{a\in A_i,t} \|z_a - h_{c(a)}(s)\|^2_{R_a^{-1}}.$$

At iterate `s^k`, `A=(P^-)^-1+sum H^T R^-1 H`, `b=sum H^T R^-1 (z-h(s^k))-(P^-)^-1(s^k-s^-)`, solve `A ds=b`; trial `s^k+ds` accepted only if the **actual nonlinear** E decreases, with clear termination, numerical failures and max-iteration statuses. Never re-predict or change the prior inside iterations. 4. Set covariance `P^+ = inverse((P^-)^-1+sum H_final^T R^-1 H_final)` as the local final-linearization Gaussian approximation. Unmatched point: `s^+=s^-; P^+=P^-`. Euclidean state: **no Lie Jacobian reset**. 5. Store all initial SOLVED trajectories, full/selected covariance and provenance for each time step. Once complete, freeze them for the rest of incremental discovery. Use chunked host/disk storage if GPU cannot hold full time history.

UKF mode (only if requested in current implementation plan): independent alternative estimator, *same* 6D state and outer loop, 13 scaled sigma points for nonlinear `h`, exact linear process prediction and `Q`, stacked independent multiview `(u,v,Z)` measurements, stable covariance update. Do not mix EKF and UKF updates or call a global smoother. Prefer a complete, correct IEKF implementation before optional UKF parity.

8. INDIVIDUAL UNMATCHED POINT -> BIDIRECTIONAL FULL-SEQUENCE OPTIMIZATION

The sole trigger for new geometry is one raw **valid unmatched** RGB-D observation at `t_d`, camera `c`, pixel `u` from the backward scan. This is a physical point seed, **not** an already-estimated motion candidate. No pre-existing trajectory, group label, or supported-trajectory verification is required *before* estimating its motion.

8.1 Seed and unique identity

- Allocate `j`, source observation ID `(t_d,c,u)` and measured world XYZ from actual depth. Initialize `s_j,t_d=[y_world;0]`, `P_j,t_d=diag(Sigma_y,P_v0)` with appropriately uncertain velocity; as-

similate anchor data **once**, accounting for whether the chosen initial `Sigma_y` already represents the anchor sensor measurement (avoid using it twice as a likelihood).
- Reserve the seed observation while running the optimizer. The old population, including its states, covariances, source IDs, and assignments, is read-only.

8.2 Backward branch, frame by frame

- From the **anchor posterior copy** at `t_d`, for `k=t_d-1,...,0`: use `F(-dt)` and `Q_rev(dt)`; compute predicted state and covariance; project into every camera; find compatible observations that are not irrevocably owned by an old point; IEKF-correct this point alone using actual masked RGB-D. Update candidate observation claims and save states/P/innovation/visibility status per frame.
- No group-derived motion, no neighbor-trajectory averaging, no synthetic depth when occluded. If no actual observation: purely predict with growing covariance.

8.3 Forward branch, frame by frame

- **Restart from the same stored anchor posterior** (not backward terminal state) and for `k=t_d+1,...,T` run ordinary forward `F(+dt),Q_f`, matching/updating this point alone. Do not assimilate the anchor observation again. Save its separate branch histories.
- Join backward results for `t<t_d`, exactly one anchor state at `t_d`, and forward results for `t>t_d`. This is a pointwise **two-sided anchored filter construction**, **not** a full-sequence RTS posterior nor an average of duplicate measurements. Do not claim otherwise.

8.4 Commit new SOLVED point, then repeat

- On completed numerical optimization, insert `j` and its entire estimated temporal state sequence in SOLVED **immediately**. No extra holdout validation pass, no arbitrary minimum number of temporal matches, no global reoptimization and no "candidate commitment" concept beyond basic record persistence.
- Keep per-frame `REAL_MEASUREMENT` versus `PREDICTION_ONLY` flags and covariance. A single-observation point has a trajectory of uncertain predictions and must not falsely cover unrelated foreground observations merely because its predicted position is nearby.
- Match/reconcile existing raw-observation IDs without changing old tracks. For exactly duplicated seed/source observations, do not create a second identity. New point consumes the observations actually associated with it; no unchecked bulk claiming of proximity pixels.
- Restart entire backward scan at `T` after each committed addition. Previously explored but still unclaimed observations remain eligible if the model changes. A numerically failed seed enters a `(observation_id, model_version, failure_reason)` registry to avoid an infinite identical retry; changing the model version can permit retry.

9. EXACT HIGH-LEVEL CONTROLLER (PSEUDOCODE)

```
# Names below are normative APIs to implement, not claims of existing functions.
def run_exact_pointwise_discovery(dataset, config):
    data = load_calibrated_masked_rgbd(dataset, config)
    source_obs = index_raw_valid_foreground_observations(data)

    solved, ownership = initialize_ALL_frame0_foreground_points(source_obs, data)
    traj = run_initial_large_forward_iekf_or_ukf(solved, data, ownership, config)
    traj.freeze_all_existing()       # BOTH states AND covariance arrays immutable
    report_initial_tracking(traj, ownership)

    attempted = set()                # (raw_obs_id, solved_model_version)
    version = 0
```

```
    events = []
    while True:
        match = match_all_frozen_solved_to_real_foreground(
            data, traj, ownership, config)
        seed_ref = first_unmatched_in_reverse_frame_camera_pixel_order(
            match, source_obs, attempted, version)
        if seed_ref is None:
            break
        seed = initialize_one_point_at_actual_discovery_frame(seed_ref, data)
        reserve_exactly_one_source_observation(seed_ref, seed.point_id)
        anchored_state = seed.initial_6d_posterior()  # no duplicate anchor
↪   likelihood
        backward = track_one_point_frames(
            seed.point_id, anchored_state,
            frames=range(seed.frame - 1, -1, -1),
            direction='backward', frozen_solved=traj, data=data,
            ownership=ownership, config=config)
        forward = track_one_point_frames(
            seed.point_id, anchored_state,
            frames=range(seed.frame + 1, data.T + 1),
            direction='forward', frozen_solved=traj, data=data,
            ownership=ownership, config=config)
        new_track = join_branches_exactly_once(backward, anchored_state, forward)
        if new_track.has_numerical_failure:
            release_temporary_claims(seed.point_id)
            attempted.add((seed_ref, version))
            write_FAILURE_debug(seed, new_track)
            continue
        # No special candidate validation. Geometry/trajectory is now SOLVED.
        commit_new_solved_point_and_observation_ids(seed, new_track, ownership)
        assert_old_trajectories_bitwise_unchanged(traj)
        traj.append_new_point(new_track)
        version += 1
        write_event_debug_and_actual_2D_3D_visualization(seed, new_track, traj)
        # This loop AUTOMATICALLY RESTARTS the reverse scan at frame T.
    return save_final_results(solved, traj, ownership, events, config)
```

A correct implementation may cache/recompute the matching index incrementally, but its output must be equivalent to the above authoritative full reverse rescan. Have a test comparing incremental vs brute-force scan.

10. REQUIRED NUMERICAL + VISUAL DEBUG (BLOCKING DELIVERABLE)

Create `output/<scene>/pointwise_rgbd/` with:

```
run_manifest.json                  (commit, config, calibration hash, seed,
↪  state/noise conventions)
initial_forward/summary.json
initial_forward/trajectory_store/* (chunked, stable point IDs and time indices)
initial_forward/nis_by_frame.csv
initial_forward/matching_histogram.json
initial_forward/diagnostics.html
initial_forward/trajectory_viewer.html
raw_observations/ownership_index.*
discovery/events.jsonl             (one event per new seed; commit/failure reason)
```

```
discovery/matching_by_frame.csv   (raw/matched/unmatched/invalid/conflicts)
discovery/nearest_residuals.csv   (per-frame quantiles + empirical separation)
discovery/events/<event_id>/seed.json
.../state_by_frame.csv             (frame,time,x,y,z,vx,vy,vz,observed,P
 ↪  trace,minEig,NIS)
.../associations.csv              (original frame,camera,pixel,z,r,S,accepted/reason)
.../trajectory.png                (XYZ and speed vs time, actual observation markers)
.../covariance_nis.png             (uncertainty, residual/NIS, prediction-only
 ↪  intervals)
.../image_overlay_<frame>_<cam>.png (real RGB, seed/reprojection/accepted/rejected
 ↪  matched pixels)
.../depth_residual_<frame>_<cam>.png (metric residual where valid, never synthetic
 ↪  depth)
.../local_3d.html                 (interactive old frozen vs new track + actual
 ↪  measured points)
discovery/geometry_growth.html   (interactive frame slider and event slider)
final/summary.json
final/point_tracks.*
final/unexplained_observations.*
```

Numerical metrics (never just averages): matched/unmatched and duplicate observation counts; residual **distributions** (median, p90/p95/p99 and histograms) both metric and innovation-normalized, gap/overlap where measured; NIS over directly observed updates; P min eig/trace/condition, PSD repairs and posterior carry-forward checks; real vs predicted-only frame counts; discovery frame/anchor, every association/rejection reason; change in explained foreground per committed point; total SOLVED growth; time for projection, kNN, IEKF iterations, image I/O, rebuilding search index; numerical solver statuses; complete original timestamps/frame IDs. Units clearly labeled.

Visual: Actual RGB+mask with old model reprojection in one color, seed observation uniquely highlighted, new point prediction vs real match in different symbols; actual depth residual heatmaps; interactive HTML 3D time slider showing ALL or spatially sampled *display-only* frozen old points and EVERY selected new track with measured-vs-predicted markers; event-by-event geometry growth; per-seed position/velocity/covariance/NIS plots. Visual subsampling is permitted for DISPLAY ONLY and must be labeled, never for inference. Fixed color identity across frames. Never publish a generic placeholder visualization and call it measured data. Include a representative screenshot or static preview of the real generated HTML.

Ensure logs are bounded via streaming and previews while retaining exact raw per-point provenance; use file-backed history for large runs.

11. TESTS THAT MUST PASS

Math/unit:

1. Pinhole backprojection/projection round trip with calibrations and metric depth; finite-difference 3x6 H including camera rotation, depth convention and distortion if supported.
2. `F(+dt)F(-dt)=I`, symmetry/PSD for `Q_f`, `Q_rev`; quadratic identity `e_f^T Q_f^-1 e_f == e_rev^T Q_rev^-1 e_rev` on random states, including anisotropic noise.
3. IEKF linear measurement equals exact Gaussian KF; nonlinear image projection energy decreases on accepted steps; all covariances propagated (none reset to P0); P PSD and no false overconfidence in prediction-only runs.
4. Same physical RGB-D observation cannot be counted twice via XYZ and `(u,v,Z)` or via two branches at anchor.
5. No Lie-group rotation reset anywhere in 6D point filter; state remains `[world_xyz, world_velocity]`.

Algorithm/integration:

6. Initial frame includes all valid foreground raw observations (up to explicitly reported geometric fusion), no synthetic background depth, no GT labels in estimation.
7. After one new seed, old states/covariances **bitwise identical**; per-point backward frames exactly `t_d-1...0`, forward `t_d+1...T`; one anchor; exactly one point optimizer called for that seed.
8. One entire rigid moving part invisible at frame 0 but visible later: first real later observation is seeded; newly discovered points can be tracked independently without any group transform or K variable.
9. Static scene, translation, rotating object with curved point tracks, temporarily occluded/reappearing point, multi-camera overlap, known depth holes; distinguish real observation vs predicted-only states.
10. One-to-one raw-observation ownership; exact duplicate seed skipped; failure marked attempted at same model version, no infinite retry, retries allowed after version increment.
11. Deterministic `T -> 0` unmatched discovery ordering and restart from T after every successful addition; compare cached/incremental search to full brute force.
12. Initial big optimization runs **once**; for M new points, only M independent point optimizations run, NEVER rerun initial old filter.
13. IEKF/UKF parity on linear synthetic data if UKF mode implemented; UKF must not share IEKF updates.
14. Test debug output files exist, are readable, numeric plots derive from recorded data, HTML 3D viewer has temporal controls and real measured frames, valid paths; no placeholders.
15. Memory regression: chunked history and streaming do not scale as dense `N*T*C*H*W` nor allocate an all-frame rasterizer; record peak GPU memory and performance at representative scene scale.

Real dataset smoke: at least one genuine scene, at least 3 frames and 2 cameras, successful source calibration parsing, initial big run and a real discovery event (if scene contains unmatched observations), saved numeric + visual debug. If no discovery occurs, report truthfully and run a deterministic synthetic hidden-part fixture for the event path. Do not fabricate success. Run full scene after smoke where hardware/time permits, report exact completed fraction and blocker if not.

12. IMPLEMENTATION MODULE CONTRACT

Choose actual naming adapted to repository conventions, keeping responsibility boundaries:

```
pointwise_rgbd/
  config.py                (validated physical units / defaults / CLI)
  dataset.py               (chronological cameras/raw RGB,D,M,V and timestamps)
  projection.py            (world<->camera, pinhole/distortion, Jacobians)
  observations.py          (raw IDs, backprojection, covariance, strict masks)
  state6.py                (6D state + full covariance, stable point ids)
  process6.py              (F, Qf, inverse factor/Frev/Qrev)
  association.py           (kNN+projection+visibility+ownership+residual audit)
  filter_iekf.py           (single-point and vectorized IEKF with true MAP
 ↪  acceptance)
  filter_ukf.py            (optional independent alternative, SAME 6D state)
  store.py                 (chunked temporal tracks, immutable frozen old state)
  discovery.py             (one seed at a time, backward/forward branches)
  controller.py            (initial once -> reverse scan -> one point -> restart)
  diagnostics.py           (number tables, per-seed plots, actual visual outputs)
  visualization.py         (2D overlay, interactive 3D and event timeline)
tools/run_pointwise_rgbd_discovery.py

tests/test_pointwise_projection.py
...
tests/test_pointwise_discovery_controller.py
...
```

CLI contract (implement actual path arguments and `--help`):

```
python tools/run_pointwise_rgbd_discovery.py \
  --scene /PATH/TO/SCENE \
  --output /PATH/TO/OUTPUT \
  --estimator iekf \
  --all-cameras \
  --all-frames \
  --debug-numeric \
  --debug-visual
```

For scalable local debugging support optional `--max-events`, `--frame-limit`, `--camera-limit` **as explicit tests only**; full run defaults must not silently truncate observations/frames/seed candidates. Also support deterministic `--seed` and resumable atomic checkpointing. Make output versioned/isolated to avoid overwriting prior working 3DGS data.

13. DELIVERY ORDER AND DEFINITION OF DONE

1. Repository audit + verified reuse map (include actual files/line locations, current git commit).
2. Pure CPU numeric reference implementation and tests for state, process, reverse factor, projection, IEKF.
3. GPU vectorized initial forward IEKF for *all* frame-0 points, with deterministic associations and covariance carry-forward.
4. Frozen-track store + exact raw-observation ownership and reverse unmatched search.
5. One-seed bidirectional filter from discovery frame, immediate addition + restart; explicitly no group clustering.
6. Mandatory numeric debug and actual-image / interactive-3D visual debug, with deterministic synthetic fixtures.
7. Performance improvements with equivalence tests against the reference logic; checkpoint/resume.
8. Real-scene smoke, full-scale run when possible, detailed report with measured numbers and links to artifacts.

Final Codex report MUST show exactly: files changed, equations/representation implemented, `N0`, number of new point seeds optimized, final SOLVED size, first/last frame, camera count, number of actual observation matches and unmatched raw pixels, per-stage runtime and memory, failure/ambiguity counts, passing/failing tests, paths to actual numeric/visual debug. Never say “implemented exactly” if any stage remains a stub or untested.

14. PRIOR DOCUMENTS AND PRECEDENCE

This 22 October 2026 context **supersedes** prior fixed-K point-motion specifications (21 October 2026, v3), full-window per-part SE(3) IEKS proposals, and the 3DGS 12D orientation-state implementation **where they conflict** with the above point-only discovery algorithm. Retain their transferable RGB-D mask/validity, camera calibration, numerical covariance, and sparse GPU implementation principles. Maurer et al., arXiv:2506.01514v1, is relevant when estimating a *Lie-group pose* in a later rigid-group stage, **not a reason to introduce an SE(3) rotation state or Lie covariance reset into the current six-dimensional Euclidean point tracker**.

Final non-negotiable invariant: ONE initial large `0->T` point-tracking optimization. Thereafter EACH newly encountered unmatched valid RGB-D observation seeds its OWN `t_d->0` plus `t_d->T` 6D state optimization; ALL previously SOLVED states and covariances remain immutable; append the newly estimated point, restart missing-point search at T; produce inspectable true numeric and visual debug.